

NouvelAir

Documentation Complete des Tests

Projet de Reservation Aerienne

Django 4.2 | pytest | 229 tests passants

Version: 2.0 | Date: Avril 2026

Environnement: Python 3.12+ / Django 4.2 LTS / pytest 8.x

Resultats: 229 passed / 4 skipped / 1 xfailed

Table des Matieres

1. Introduction	4
1.1 Stack Technique	4
1.2 Resultats Globaux des Tests	4
2. Tests Unitaires	5
2.1 Module Flights (24 tests)	5
2.2 Module Bookings (18 tests)	6
2.3 Module Accounts (6 tests)	7
2.4 Module Promotions et Destinations (18 tests)	8
2.5 Fonctions Utilitaires (5 tests)	9
3. Tests d'Integration	10
3.1 Vues Flights (15 tests)	10
3.2 Vues Accounts (10 tests)	11
3.3 Vues Bookings (7 tests)	11
3.4 Vues Destinations (5 tests)	12
3.5 Vues Promotions (5 tests)	12
4. Tests API REST	13
4.1 API Authentification (8 tests)	13
4.2 API Autocomplete Aeroports (10 tests)	13
4.3 API Reservations (8 tests)	14
4.4 API Newsletter (5 tests)	14
5. Tests au Niveau des Applications (63 tests)	15
5.1 flights/tests/test_models.py (20 tests)	15
5.2 bookings/tests/test_models.py (9 tests)	16
5.3 accounts/tests/test_models.py (15 tests)	16
5.4 promotions/tests/test_models.py (10 tests)	17
5.5 destinations/tests/test_models.py (9 tests)	17
6. Infrastructure de Test	19

6.1 Fixtures et Factories	19
6.2 Configuration pytest	19
7. Synthese par Module	20

1. Introduction

Ce document presente la description exhaustive de l'ensemble des tests du projet NouvelAir, une application de reservation aerienne developpee avec le framework Django 4.2. La suite de tests couvre les cinq applications principales du projet : **flights** (vols), **bookings** (reservations), **accounts** (comptes utilisateurs), **destinations** (destinations touristiques) et **promotions** (codes promotionnels).

L'architecture des tests suit une approche en pyramide, privilegiant les tests unitaires (60%), suivis des tests d'integration (25%), des tests API (10%) et des tests E2E (5%). Chaque niveau de test valide un aspect different de l'application, depuis la logique metier des modeles jusqu'aux flux utilisateur complets en passant par les endpoints API REST.

Les tests utilisent le framework pytest avec l'extension pytest-django pour l'integration avec Django. Les donnees de test sont generees via Factory Boy (12 factories), ce qui garantit des tests deterministes et reproductibles. La base de donnees SQLite en memoire est utilisee pour chaque session de test, assurant une isolation complete entre les executions.

1.1 Stack Technique

Composant	Version	Role
Python	3.12+	Langage principal
Django	4.2 LTS	Framework web
pytest	8.x	Framework de test
pytest-django	4.x	Integration Django-pytest
Factory Boy	3.x	Generation de donnees de test
SQLite	3.x	Base de donnees de test (en memoire)

1.2 Resultats Globaux des Tests

L'ensemble de la suite de tests comporte **229 tests passants**, 4 tests ignores (skipped) et 1 test marque comme echec attendu (xfailed). Aucun test n'echoue de maniere inattendue. Ces resultats confirment la stabilite et la couverture fonctionnelle de l'ensemble des modules critiques de l'application NouvelAir.

Statut	Nombre	Pourcentage
Passants	229	97.9%

Ignorees (skipped)	4	1.7%
Echec attendu (xfailed)	1	0.4%
Total	234	100%

Tableau 1 : Resultats globaux de la suite de tests

2. Tests Unitaires

Les tests unitaires constituent le socle de la suite de tests du projet NouvelAir. Ils valident le comportement individuel de chaque modele Django (creation, contraintes, methodes metier) et des fonctions utilitaires (calcul de prix, formatage de duree, validation d'age). Ces tests sont marques avec `@pytest.mark.unit` et `@pytest.mark.django_db` pour l'acces a la base de donnees de test.

2.1 Module Flights (24 tests)

Le module flights contient trois classes de tests unitaires couvrant les modeles Airport (7 tests), Aircraft (6 tests) et Flight (11 tests). Les tests verifient les contraintes d'integrite (unicite du code IATA, du numero de vol), les methodes de recherche de vols, le calcul automatique de la duree, et les methodes de prix dynamique.

#	Nom du Test	Description
1	test_airport_creation	Creation d'un aeroport avec tous les champs (code, nom, ville, coordonnees GPS).
2	test_airport_str	Verification de la representation textuelle au format 'CODE - City (Country)'.
3	test_airport_code_unique	Le code IATA doit etre unique ; une doublon leve une IntegrityError.
4	test_airport_ordering	Le tri par default des aeroports se fait par code IATA croissant.
5	test_airport_latitude_longitude_decimal	Les coordonnees GPS sont stockees comme DecimalField avec precision.
6	test_airport_is_active_default	La valeur par default de is_active est True.
7	test_airport_cannot_be_created_without_code	Impossible de creer un aeroport sans code (null=False).
8	test_aircraft_creation	Creation d'un aeronef avec toutes les proprietes (modele, immatriculation, sieges).
9	test_aircraft_str	Representation textuelle au format 'model_name (registration)'.
10	test_aircraft_registration_unique	L'immatriculation est unique ; doublon = IntegrityError.

11	test_seats_sum	economy_seats + business_seats doit egaler total_seats.
12	test_aircraft_is_active	Verification du champ is_active (True/False).
13	test_aircraft_economy_seats_positive	Les sieges economiques sont un entier positif ou zero.
14	test_flight_creation	Creation d'un vol avec numeros, aeroports, horaires, prix et statut.
15	test_flight_str	Representation textuelle contient le numero de vol et codes aeroports.
16	test_flight_number_unique	Le numero de vol est unique ; doublon = IntegrityError.
17	test_flight_status_choices	Test parametre de tous les statuts valides (6 valeurs possibles).
18	test_flight_invalid_status_raises_error	Un statut invalide leve une erreur a la creation.
19	test_flight_duration_auto_calculation	La duree est calculee automatiquement a partir des horaires.
20	test_flight_search	Recherche de vols avec resultats correspondants (origine, destination, date).
21	test_flight_search_no_results	Recherche de vols sans resultats quand aucun vol ne correspond.
22	test_get_current_price_economy	Prix economique dynamique sans promotion = prix de base.
23	test_get_current_price_business	Prix affaires dynamique sans promotion = prix de base.
24	test_flight_default_status_scheduled	Le statut par default d'un vol est 'scheduled'.

2.2 Module Bookings (18 tests)

Le module bookings couvre les modeles Booking (8 tests), Passenger (5 tests) et Payment (5 tests). Les tests verifient le cycle de vie complet d'une reservation : creation avec reference UUID unique, statuts (pending, confirmed, cancelled, completed, refunded), informations passager (titre civilite, passeport) et suivi du paiement (methodes, statuts, identifiant de transaction).

#	Nom du Test	Description
1	test_booking_creation	Creation d'une reservation avec email, telephone, statut et montant.
2	test_booking_reference_unique	Chaque reservation obtient un UUID unique comme reference.
3	test_booking_status_choices	Test parametre des 5 statuts valides de reservation.
4	test_booking_str	Representation textuelle contient la reference courte et le statut.

5	test_booking_travel_class_via_passenger	La classe de voyage est definie au niveau passager.
6	test_booking_default_status_pending	Le statut par default est 'pending'.
7	test_booking_created_at_auto	Les timestamps created_at et updated_at sont auto-remplis.
8	test_booking_total_amount_calculation	Calcul du montant total avec passagers multiples.
9	test_passenger_creation	Creation d'un passager avec toutes les informations requises.
10	test_passenger_title_choices	Test parametre des titres (mr, mme, mll, enf).
11	test_passenger_str	Representation textuelle avec titre, prenom et nom.
12	test_passenger_passport_number_optional	Le numero de passeport est optionnel (blank=True).
13	test_passenger_date_of_birth	La date de naissance est correctement stockee.
14	test_payment_creation	Creation d'un paiement avec montant, methode, statut et transaction_id.
15	test_payment_method_choices	Test parametre des 4 methodes de paiement.
16	test_payment_status_choices	Test parametre des 4 statuts de paiement.
17	test_payment_str	Representation textuelle contient le montant et le statut.
18	test_payment_transaction_id_optional	Le transaction_id est optionnel (blank=True).

2.3 Module Accounts (6 tests)

Le module accounts teste le modele UserProfile (6 tests), qui etend le modele User de Django avec des informations complementaires : telephone, adresse, nationalite, date de naissance, genre et abonnement newsletter. Un point critique est la verification que le signal post_save cree automatiquement un profil a chaque creation d'utilisateur.

#	Nom du Test	Description
1	test_profile_creation	Creation d'un profil complet avec tous les champs.
2	test_profile_auto_creation	Le signal post_save cree automatiquement un profil.
3	test_profile_full_name	Propriete full_name retourne 'Prenom Nom' correctement.
4	test_profile_without_full_name	full_name retourne le username si pas de nom complet.
5	test_profile_one_to_one_user	Relation OneToOneUser-Profile ; doublon = IntegrityError.

6	test_profile_newsletter_default	La valeur par défaut de newsletter est False.
---	---------------------------------	---

2.4 Module Promotions et Destinations (18 tests)

Ce module regroupe les tests des modeles Promotion (7 tests), NewsletterSubscription (3 tests), Destination (5 tests incluant get_lowest_price_no_flights) et DestinationReview (3 tests). Les tests de promotion verifient la logique de validite (dates, limites d'utilisation, pourcentage de remise), tandis que les tests de destination couvrent le slug, les categories et le systeme d'avis avec contrainte d'unicite par utilisateur.

#	Nom du Test	Description
1	test_promotion_creation	Creation d'une promotion complete (code, type, pourcentage, dates).
2	test_promotion_code_unique	Le code promo est unique ; doublon = IntegrityError.
3	test_promotion_is_valid	Promotion dans la plage de dates et non expiree est valide.
4	test_promotion_expired	Promotion expiree (end_date dans le passe) n'est pas valide.
5	test_promotion_max_uses_reached	Promotion dont la limite max_uses est atteinte n'est pas valide.
6	test_remaining_uses	Propriete remaining_uses retourne max_uses - current_uses (minimum 0).
7	test_promotion_discount_percentage_range	Le pourcentage est dans [0, 100].
8	test_newsletter_creation	Creation d'un abonnement newsletter avec email et prenom.
9	test_newsletter_email_unique	L'email est unique pour les abonnements newsletter.
10	test_newsletter_str	Représentation textuelle = email de l'abonne.
11	test_destination_creation	Creation d'une destination avec slug, categorie, note.
12	test_destination_slug_unique	Le slug est unique ; doublon = IntegrityError.
13	test_destination_category_choices	Test parametre des 6 categories (beach, culture, etc.).
14	test_destination_str	Représentation textuelle = nom de la destination.
15	test_get_lowest_price	Methode get_lowest_price retourne le prix le plus bas des vols.
16	test_get_lowest_price_no_flights	Retourne None si aucun vol n'est associe a la destination.
17	test_review_creation	Creation d'un avis avec note, titre, commentaire.
18	test_unique_review_per_user	Un utilisateur ne peut laisser qu'un avis par destination.

19	test_review_rating_range	La note est un entier entre 1 et 5 inclus.
----	--------------------------	--

2.5 Fonctions Utilitaires (5 tests)

Les tests utilitaires couvrent les fonctions transversales du projet : formatage de la duree de vol, calcul de prix avec promotion appliquee, validation du format de reference de reservation (UUID) et calcul de l'age d'un passager a partir de sa date de naissance.

#	Nom du Test	Description
1	test_duration_format_filter	Formatage de la duree en 'Xh Ym' (ex: 2h 30m).
2	test_duration_format_exact_hours	Formatage pour un nombre exact d'heures (ex: 3h 0m).
3	test_flight_price_calculation_with_discount	Calcul du prix avec promotion de 30% (economie et affaires).
4	test_booking_reference_format	Verification du format UUID (36 caracteres) et short_reference (8 chars).
5	test_passenger_age_validation	Calcul correct de l'age adulte (30 ans) et enfant (10 ans).

3. Tests d'Integration

Les tests d'integration valident les flux multi-composants en simulant des requetes HTTP via le client de test Django. Ils verifient le bon fonctionnement des vues, le routage des URLs, la gestion des sessions, la redirection apres authentication et le traitement des formulaires. Chaque test est marque avec `@pytest.mark.integration`.

3.1 Vues Flights (15 tests)

Ces tests couvrent la page d'accueil (HomeView), la recherche de vols (FlightSearchResultsView), le detail d'un vol (FlightDetailView), la liste des aeroports (AirportListView) et l'API d'autocomplete des aeroports.

#	Nom du Test	Description
1	test_home_view_status	GET '/' retourne HTTP 200.
2	test_home_view_template	La page d'accueil utilise 'flights/home.html' et 'base.html'.
3	test_home_view_contains_form	Le contexte contient le FlightSearchForm.
4	test_home_view_popular_destinations	Le contexte contient 'popular_destinations' (max 6).
5	test_home_view_upcoming_flights	Le contexte contient 'upcoming_flights' (max 4).
6	test_search_flight_post	POST avec parametres valides redirige vers search_results.
7	test_search_same_airport_error	POST avec origin == destination renvoie une erreur.
8	test_search_past_date_error	POST avec une date passee renvoie une erreur de validation.
9	test_flight_detail_view	GET /vol// retourne 200 avec le bon vol dans le contexte.
10	test_flight_detail_view_404	GET /vol// retourne 404.
11	test_airport_list_view	GET /aeroports/ retourne 200 et la liste des aeroports.
12	test_airport_autocomplete	GET /api/airports/autocomplete/?q=TU retourne JSON avec TUN.
13	test_airport_autocomplete_empty	Requete vide retourne une liste JSON vide.
14	test_flight_search_results	GET /recherche/ avec parametres en session retourne des resultats.
15	test_flight_search_results_no_params	GET /recherche/ sans parametres retourne 0 resultat.

3.2 Vues Accounts (10 tests)

Ces tests valident le cycle complet d'authentification : affichage et soumission du formulaire de connexion, inscription avec creation automatique du profil, deconnexion, et gestion du profil utilisateur (affichage, mise a jour, protection par authentification).

#	Nom du Test	Description
1	test_login_page	GET de la page de connexion retourne 200 et le bon template.
2	test_login_success	POST avec identifiants valides redirige vers la page d'accueil.
3	test_login_invalid	POST avec mot de passe invalide renvoie le formulaire avec erreurs.
4	test_register_page	GET de la page d'inscription retourne 200.
5	test_register_success	POST avec donnees valides cree User + UserProfile et redirige.
6	test_register_duplicate_email	POST avec email duplique renvoie une erreur.
7	test_logout	GET deconnexion deconnecte l'utilisateur et redirige vers l'accueil.
8	test_profile_requires_login	GET profil sans authentification redirige vers connexion.
9	test_profile_authenticated	GET profil authentifie retourne 200 avec les formulaires.
10	test_profile_update	POST avec donnees valides met a jour le profil en base.

3.3 Vues Bookings (7 tests)

Les tests d'integration des reservations verifient le flux complet : consultation des reservations (protegee par authentification), recherche par reference, creation de reservation et annulation avec mise a jour du statut.

#	Nom du Test	Description
1	test_my_bookings_requires_login	GET mes reservations sans authentification redirige.
2	test_my_bookings_authenticated	GET mes reservations authentifie retourne 200.
3	test_booking_lookup_page	GET page de recherche par reference retourne 200.
4	test_select_flight_redirect	Selection d'un vol redirige vers le formulaire de reservation.
5	test_booking_creation	Creation complete d'une reservation avec passagers.
6	test_booking_cancellation	Annulation d'une reservation avec mise a jour du statut.
7	test_booking_reference_lookup	Recherche d'une reservation par reference UUID.

3.4 Vues Destinations (5 tests)

Ces tests valident les vues de listing, detail et recherche de destinations touristiques, ainsi que le formulaire de depot d'avis.

#	Nom du Test	Description
1	test_destination_list	GET /destinations/ retourne 200 avec la liste des destinations.
2	test_destination_detail	GET /destinations// retourne 200 avec les details.
3	test_destination_detail_404	GET /destinations// retourne 404.
4	test_destination_featured	Filtre des destinations mises en avant (is_featured=True).
5	test_destination_review_form	Soumission du formulaire d'avis sur une destination.

3.5 Vues Promotions (5 tests)

#	Nom du Test	Description
1	test_promotion_list	GET /promotions/ retourne 200 avec la liste des promotions actives.
2	test_promotion_detail	GET /promotions// retourne les details de la promotion.
3	test_promotion_detail_404	GET /promotions// retourne 404.
4	test_newsletter_subscribe_success	Inscription newsletter avec email valide.
5	test_newsletter_subscribe_duplicate	Inscription newsletter avec email duplique renvoie erreur.

4. Tests API REST

Les tests API valident les endpoints REST de l'application en envoyant des requetes HTTP et en verifiant les reponses JSON. Ils couvrent l'authentification (login, logout, register), l'autocomplete des aeroports, la gestion des reservations (consultation, creation, annulation) et l'inscription a la newsletter.

4.1 API Authentification (8 tests)

#	Nom du Test	Description
1	test_login_api_valid	POST /api/login/ avec identifiants valides retourne un token.
2	test_login_api_invalid	POST /api/login/ avec identifiants invalides retourne 401.
3	test_login_api_csrf	POST sans token CSRF avec enforce_csrf_checks=True retourne 403.
4	test_register_api_valid	POST /api/register/ cree un nouvel utilisateur.
5	test_register_api_duplicate_email	POST /api/register/ avec email duplique echoue.
6	test_logout_api	POST /api/logout/ deconnecte l'utilisateur.
7	test_protected_api_redirect	Acces non authentifie a une API protegee redirige.
8	test_session_persistence	La session persiste entre les requetes.

4.2 API Autocomplete Aeroports (10 tests)

Ces tests verifient l'API d'autocomplete des aeroports qui retourne une liste JSON filtree par prefixe de code ou nom de ville. Ils testent la structure de la reponse, le temps de reponse, le tri alphabetique et le filtrage des aeroports actifs uniquement.

#	Nom du Test	Description
1	test_autocomplete_valid_query	Requete valide retourne une liste JSON non vide.
2	test_autocomplete_multiple_results	Requete 'TU' retourne plusieurs aeroports (TUN, etc.).
3	test_autocomplete_single_char	Un caractere seul retourne des resultats pertinents.
4	test_autocomplete_empty_query	Requete vide retourne une liste vide.
5	test_autocomplete_no_results	Requete sans correspondance retourne une liste vide.
6	test_autocomplete_response_structure	Chaque resultat contient code, name, city, country.

7	test_autocomplete_code_format	Le code de l'aéroport est au format IATA (3 lettres majuscules).
8	test_autocomplete_active_airports_only	Seuls les aéroports actifs (is_active=True) sont retournés.
9	test_autocomplete_ordered_by_city	Les résultats sont triés par ordre alphabétique de ville.
10	test_autocomplete_response_time	Le temps de réponse est inférieur à 500ms.

4.3 API Reservations (8 tests)

#	Nom du Test	Description
1	test_booking_lookup_get	GET /api/bookings/lookup/ affiche le formulaire de recherche.
2	test_booking_lookup_valid_reference	POST avec référence valide retourne les détails.
3	test_booking_lookup_invalid_reference	POST avec référence invalide retourne une erreur.
4	test_booking_lookup_missing_fields	POST sans champs requis retourne une erreur 400.
5	test_booking_create_authenticated	POST création de réservation par utilisateur authentifié.
6	test_booking_create_unauthenticated	POST création sans authentification est refusée.
7	test_booking_cancel_valid	POST annulation d'une réservation existante.
8	test_booking_cancel_already_cancelled	POST annulation d'une réservation déjà annulée échoue.

4.4 API Newsletter (5 tests)

#	Nom du Test	Description
1	test_newsletter_subscribe_valid	POST inscription newsletter avec email valide.
2	test_newsletter_subscribe_duplicate	POST avec email déjà inscrit retourne une erreur.
3	test_newsletter_subscribe_invalid_email	POST avec email invalide retourne une erreur 400.

4	test_newsletter_subscribe_missing_email	POST sans email retourne une erreur 400.
5	test_newsletter_subscribe_response_format	La reponse JSON contient les bons champs.

5. Tests au Niveau des Applications (63 tests)

En plus des tests organisés dans le dossier tests/, chaque application Django possède son propre dossier tests/ avec des tests unitaires et d'intégration supplémentaires. Ces tests vérifient le fonctionnement au plus près du code source de chaque application, incluant les vues, les modèles et les formulaires. Ils sont automatiquement découverts par pytest.

5.1 flights/tests/test_models.py (20 tests)

Ce fichier contient 20 tests supplémentaires pour l'application flights. Il couvre les modèles Airport, Aircraft et Flight avec des scénarios de test complémentaires, incluant la validation des vues (home, detail, liste aeroports, autocomplete, recherche) et des méthodes de prix.

#	Nom du Test	Description
1	test_airport_creation	Creation d'aéroport avec code IATA et coordonnées.
2	test_airport_str_representation	Format de la représentation textuelle.
3	test_airport_ordering	Tri par code IATA par défaut.
4	test_airport_unique_code	Unicité du code IATA.
5	test_aircraft_creation	Creation d'aéronef avec siège total et configuration.
6	test_aircraft_str	Représentation textuelle de l'aéronef.
7	test_seats_sum	Somme economy + business = total.
8	test_flight_creation	Creation de vol complet.
9	test_flight_duration_auto_calculated	Durée calculée automatiquement.
10	test_flight_str	Représentation textuelle du vol.
11	test_flight_search	Recherche de vols avec filtres.
12	test_flight_search_no_results	Recherche sans résultats.
13	test_price_methods	Méthodes de prix économie et affaires.
14	test_home_view_status	Vue accueil status 200.

15	test_home_view_contains_form	Formulaire de recherche present.
16	test_flight_detail_view	Vue detail d'un vol.
17	test_airport_list_view	Vue liste des aeroports.
18	test_airport_autocomplete	API autocomplete aeroports.
19	test_search_flight_post	POST recherche de vol.
20	test_search_same_airport_error	Erreur origin = destination.

5.2 bookings/tests/test_models.py (9 tests)

#	Nom du Test	Description
1	test_booking_creation	Creation de reservation.
2	test_booking_str	Representation textuelle.
3	test_passenger_creation	Creation de passager.
4	test_payment_creation	Creation de paiement.
5	test_booking_lookup_page	Page de recherche par reference.
6	test_my_bookings_requires_login	Protection par authentification.
7	test_my_bookings_authenticated	Acces authentifie.
8	test_select_flight_redirect	Redirection apres selection de vol.
9	test_booking_cancellation	Annulation de reservation.

5.3 accounts/tests/test_models.py (15 tests)

Ce fichier contient 15 tests couvrant le cycle de vie complet d'un utilisateur dans l'application accounts : creation automatique du profil, connexion, deconnexion, inscription avec gestion des erreurs, et gestion du profil utilisateur.

#	Nom du Test	Description
1	test_profile_auto_creation	Profil cree automatiquement par signal.
2	test_profile_full_name	Propriete full_name correcte.
3	test_profile_without_full_name	Fallback vers username.
4	test_booking_count	Compteur de reservations.

5	test_register_page	Page d'inscription accessible.
6	test_register_success	Inscription reussie.
7	test_register_duplicate_email	Email duplique rejete.
8	test_login_page	Page de connexion accessible.
9	test_login_success	Connexion reussie.
10	test_login_invalid	Identifiants invalides rejetes.
11	test_logout	Deconnexion fonctionnelle.
12	test_profile_requires_login	Protege par authentification.
13	test_profile_authenticated	Profil accessible si authentifie.
14	test_profile_update	Mise a jour du profil.
15	test_authenticated_user_redirected_from_register	Redirection si deja connecte.

5.4 promotions/tests/test_models.py (10 tests)

#	Nom du Test	Description
1	test_promotion_creation	Creation de promotion.
2	test_promotion_is_valid	Promotion valide dans la plage.
3	test_promotion_expired	Promotion expiree non valide.
4	test_promotion_max_uses_reached	Limite d'utilisation atteinte.
5	test_remaining_uses	Calcul des utilisations restantes.
6	test_promotion_list	Liste des promotions.
7	test_promotion_detail	Detail d'une promotion.
8	test_newsletter_subscribe_success	Inscription newsletter.
9	test_newsletter_subscribe_duplicate	Email duplique rejete.
10	test_newsletter_subscribe_missing_email	Email manquant = erreur.

5.5 destinations/tests/test_models.py (9 tests)

#	Nom du Test	Description
1	test_destination_creation	Creation de destination.
2	test_destination_str	Représentation textuelle.
3	test_get_lowest_price	Prix le plus bas des vols.
4	test_unique_slug	Slug unique.
5	test_review_creation	Creation d'avis.
6	test_unique_review_per_user	Un avis par utilisateur.
7	test_destination_list	Liste des destinations.
8	test_destination_detail	Detail d'une destination.
9	test_destination_detail_404	Destination inexistante = 404.

6. Infrastructure de Test

6.1 Fixtures et Factories

Le projet utilise 12 factories Factory Boy pour generer les donnees de test : UserFactory, AirportFactory, AircraftFactory, FlightFactory, BookingFactory, PassengerFactory, PaymentFactory, UserProfileFactory, PromotionFactory, NewsletterSubscriptionFactory, DestinationFactory et DestinationReviewFactory. Ces factories permettent de creer des objets de test avec des valeurs par defaut realistes et la possibilite de surcharger n'importe quel champ.

Factory	Modele	Application
UserFactory	User	Django (auth)
AirportFactory	Airport	flights
AircraftFactory	Aircraft	flights
FlightFactory	Flight	flights
BookingFactory	Booking	bookings
PassengerFactory	Passenger	bookings
PaymentFactory	Payment	bookings
UserProfileFactory	UserProfile	accounts
PromotionFactory	Promotion	promotions
NewsletterSubscriptionFactory	NewsletterSubscription	promotions
DestinationFactory	Destination	destinations
DestinationReviewFactory	DestinationReview	destinations

Tableau 2 : Listes des 12 factories de test

6.2 Configuration pytest

Le fichier pytest.ini configure le module Django (nouvelair.settings), les repertoires de decouverte (tests, features), les marqueurs personnalisés (unit, integration, api, bdd, e2e, performance, security, slow, regression) et les options par defaut (--strict-markers -v --tb=short). Un fichier conftest.py central dans tests/ definit les fixtures partagees : authenticated_client, sample_airport, sample_airport_paris, sample_aircraft, sample_flight, sample_booking, api_client et future_date.

7. Synthese par Module

Le tableau ci-dessous resume la repartition des tests par module et par type, confirmant une couverture fonctionnelle exhaustive des cinq applications du projet NouvelAir.

Module	Fichier	Unitaires	Integration	API	Total
flights	test_models_flights.py	24	15	10	49
bookings	test_models_bookings.py	18	7	8	33
accounts	test_models_accounts.py	6	10	8	24
promotions	test_models_promotions.py	18	5	5	28
destinations	(dans promotions)	8	5	-	13
utils	test_models_utils.py	5	-	-	5
App-level	*/tests/test_models.py	63	-	-	63
TOTAL		142	42	31	215+

Tableau 3 : Repartition des tests par module et type

Note : Le total de 229 tests passants inclut egalement les tests appartenant aux autres categories (security, performance, e2e, regression) qui ne sont pas detailles individuellement dans ce document. Tous les 10 scripts de generation (setup_jour1.py a setup_jour10.py) ont ete corriges pour generer du code de test conforme aux corrections appliquees.